

Arquitetura de Computadores

Tópico 2- Organização MIPS Monociclo

- Vamos construir uma organização da arquitetura MIPS a partir deste sub-conjunto de instruções:

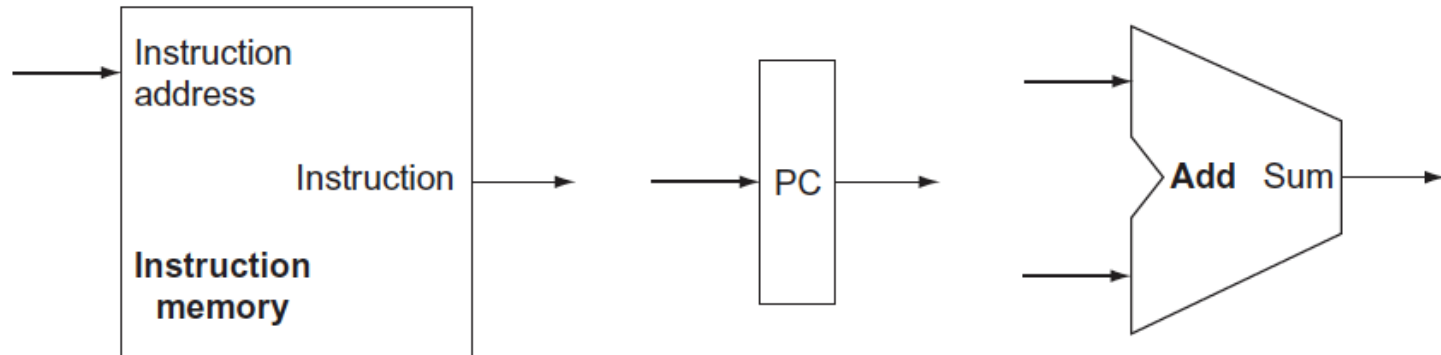
- Lógicas/Aritméticas:
add, sub
- Referência à memória:
lw, sw
- Controle de Fluxo
j, beq

Passos para Execução

1. Ler a Instrução da Memória de Instruções (**Fetch**)
Envio do conteúdo do PC para a Memória
2. Decodificar a Instrução,
Incrementar o conteúdo do PC ($PC = PC + 4$)
Ler os operandos do Banco de Registradores
3. Para instruções: ADD e SUB- Operação na ULA
Para instrução BEQ- Cálculo do endereço de desvio e comparação dos operandos
Para instruções LW e SW- Cálculo do endereço de memória de dados
4. Para instruções ADD e SUB- Escrita do resultado no Banco de Registradores
Para instruções LW e SW- Acesso (leitura ou escrita) à memória de dados
5. Para instruções ADD, SUB e LW- Escrita no banco de registradores (**Write Back**)

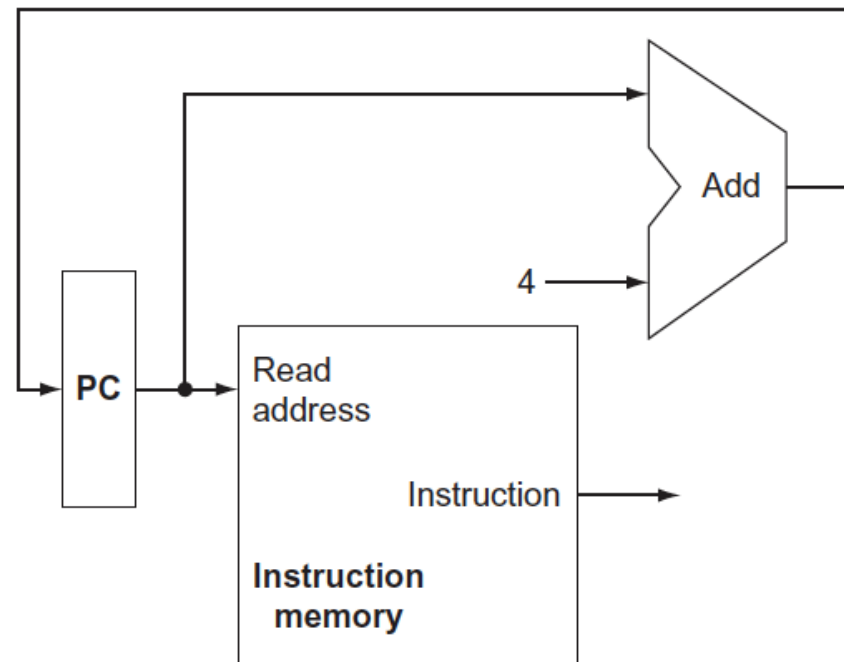
Busca de instrução

- Três elementos são necessários para executar uma busca de instrução:
 - a memória onde estão armazenadas as instruções
 - o contador de programa (PC) para armazenar o endereço da instrução
 - um somador para calcular o endereço da próxima instrução



Busca de instrução

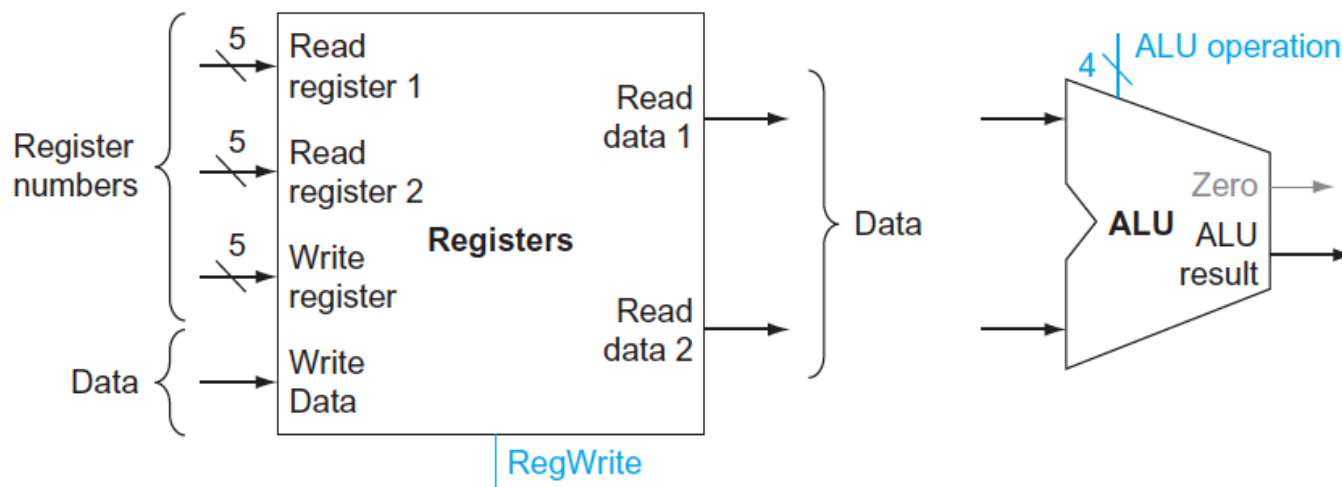
- O conteúdo do contador de programa, que contém o endereço da instrução a ser executada, é enviado para a memória de instruções
- O endereço da próxima instrução é construído a partir da soma de 4 posições ao conteúdo do contador de programa



Instruções Tipo R

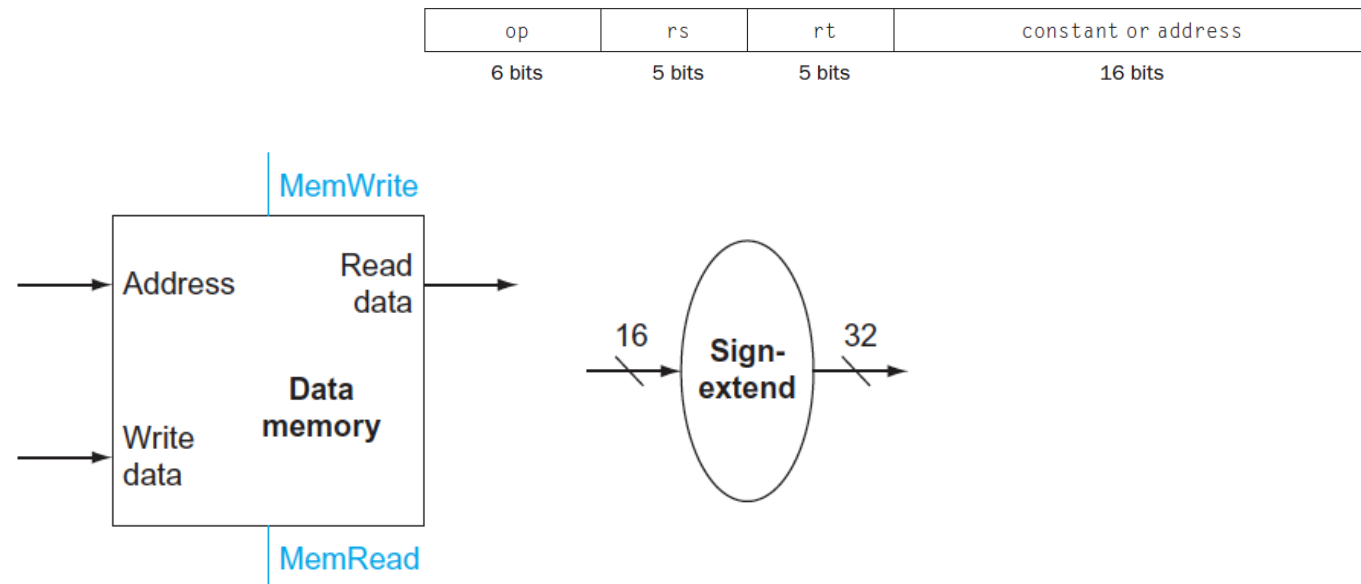
- Dois elementos são necessários para a execução de instruções Tipo R :
 - Um banco de registradores para armazenar os operandos e o resultado das operações
 - Uma Unidade Lógica/Aritmética (ALU) que será utilizada para realizar as operações

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits



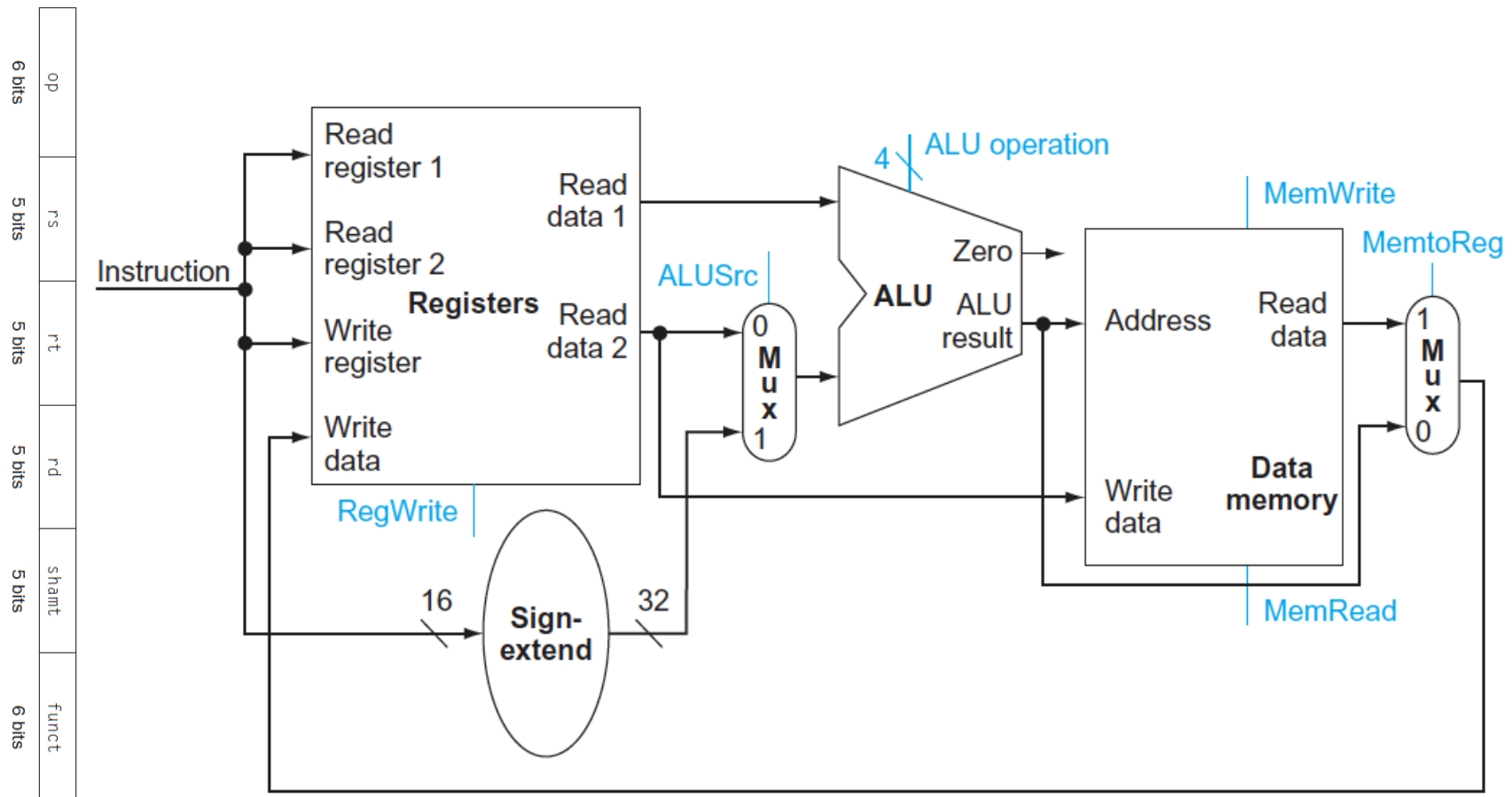
Instruções de Acesso à Memória

- Para executar instruções de acesso à memória são necessários:
 - Uma memória de dados
 - Um módulo de extensão de sinal (sign extend) para estender para 32 bits o imediato de 16 bits



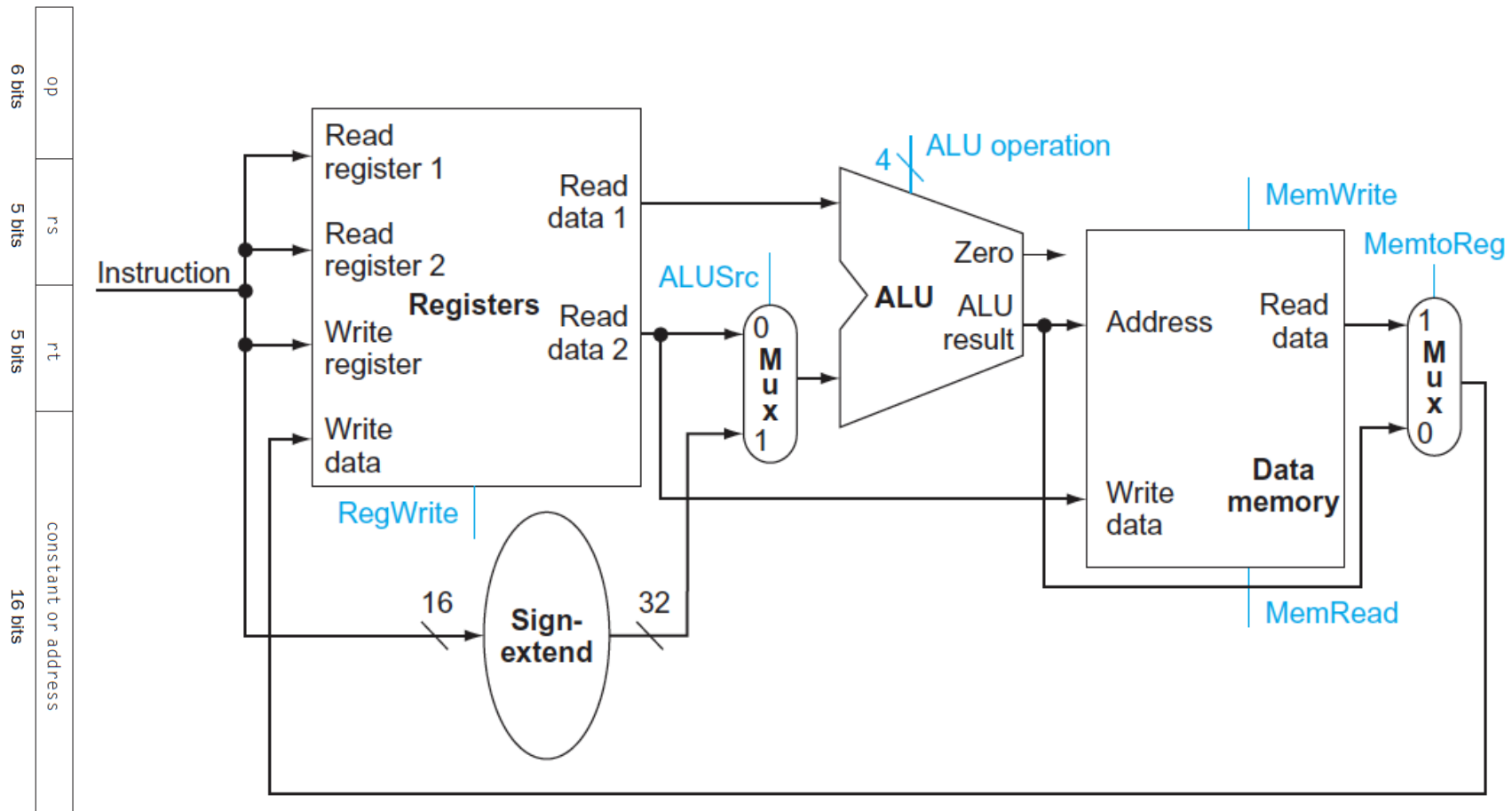
Datapath para ADD, SUB, LW, SW

- Execução para ADD e SUB ($R_d \leftarrow R_s \text{ op } R_t$)



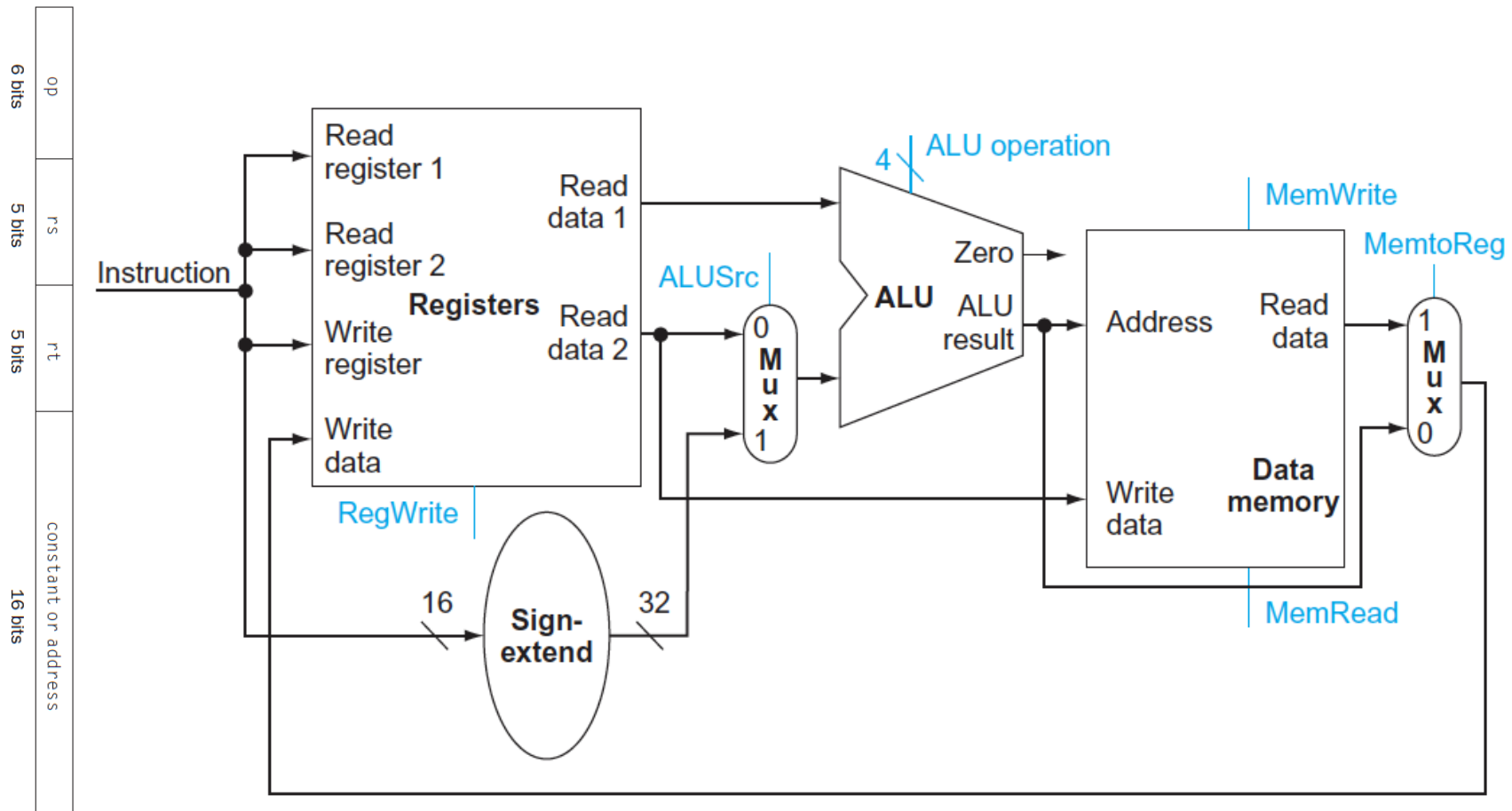
Datapath para ADD, SUB, LW, SW

- Execução para SW ($\text{Mem}[\text{est}(\text{imed16}) + \text{Rs}] \leftarrow \text{Rt}$)



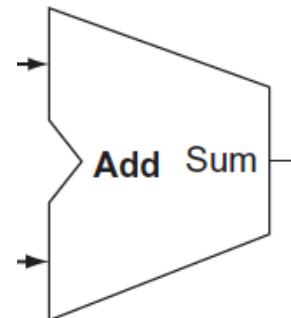
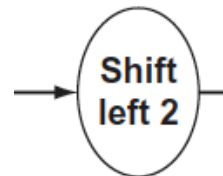
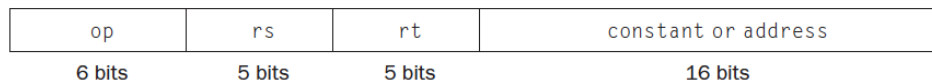
Datapath para ADD, SUB, LW, SW

- Execução para LW ($R_t \leftarrow \text{Mem}[\text{est}(\text{imed16}) + R_s]$)



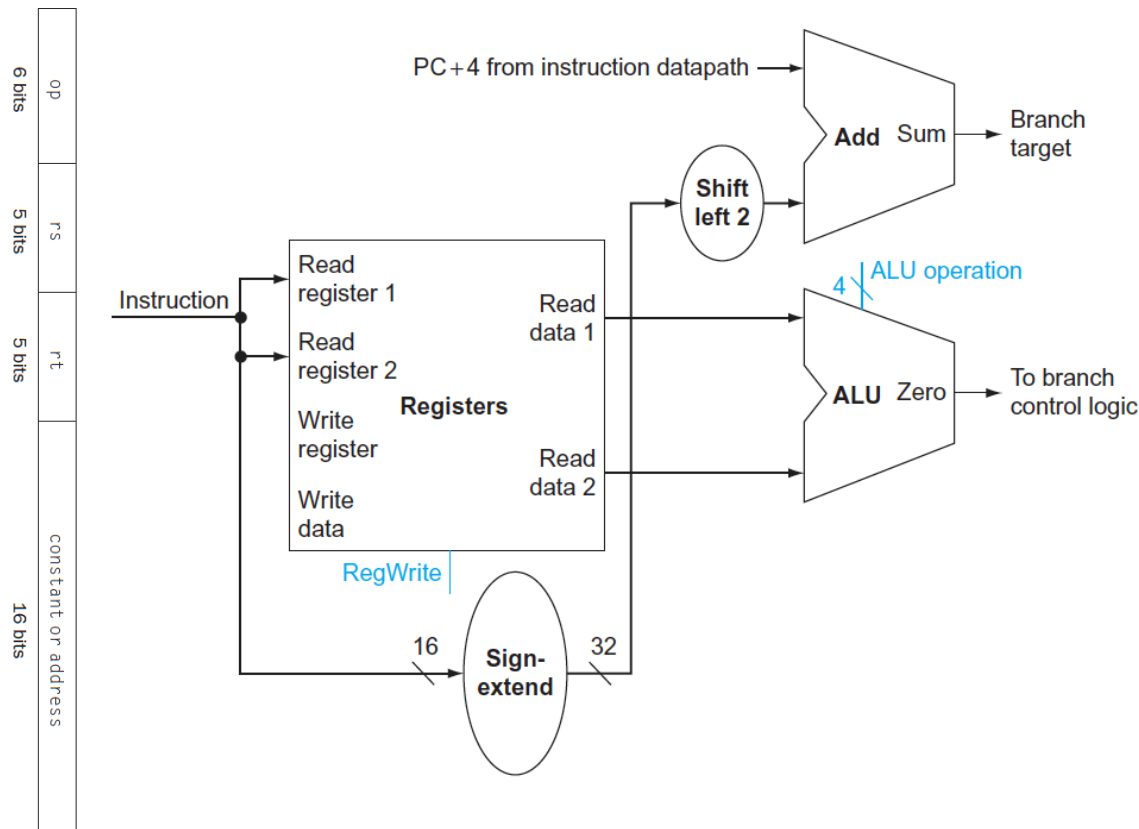
Instrução BEQ

- Para executar instrução BEQ são necessários:
 - Um deslocador de 2 bits à esquerda
 - Um somador de 32 bits



Datapath para BEQ

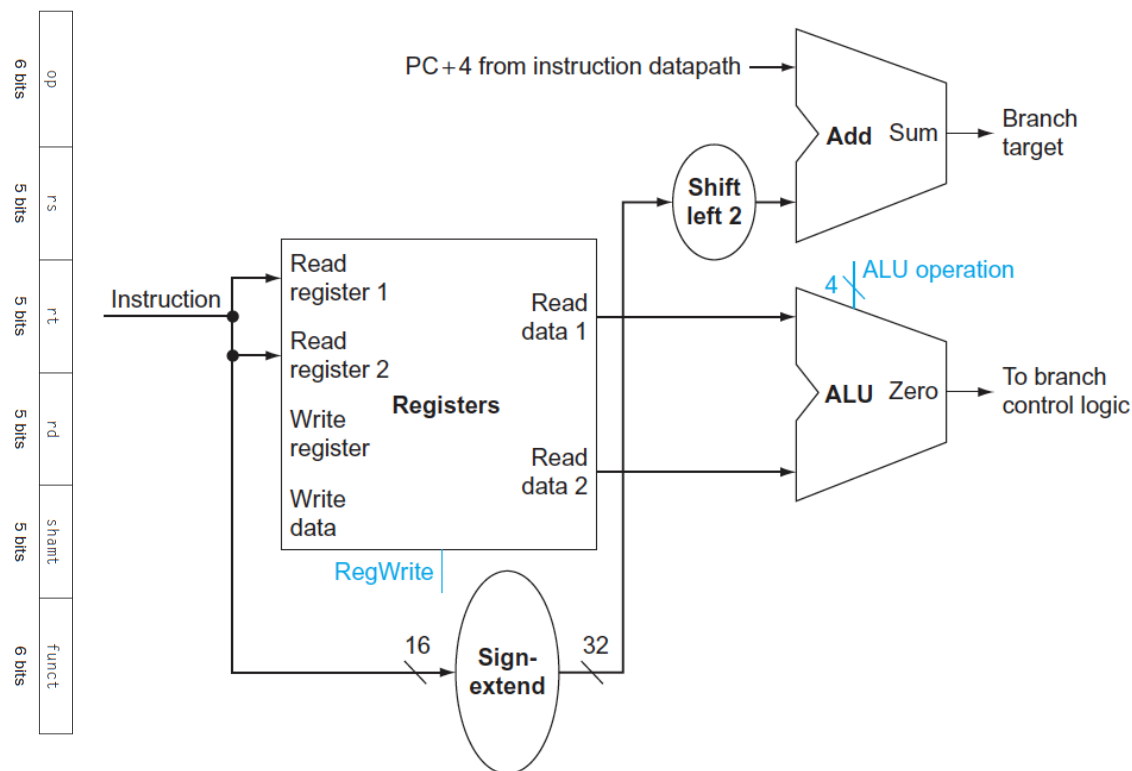
- Execução para BEQ (se $rs \neq rt$, $PC = PC + (\text{est}(\text{Imed16}) \ll 2)$)



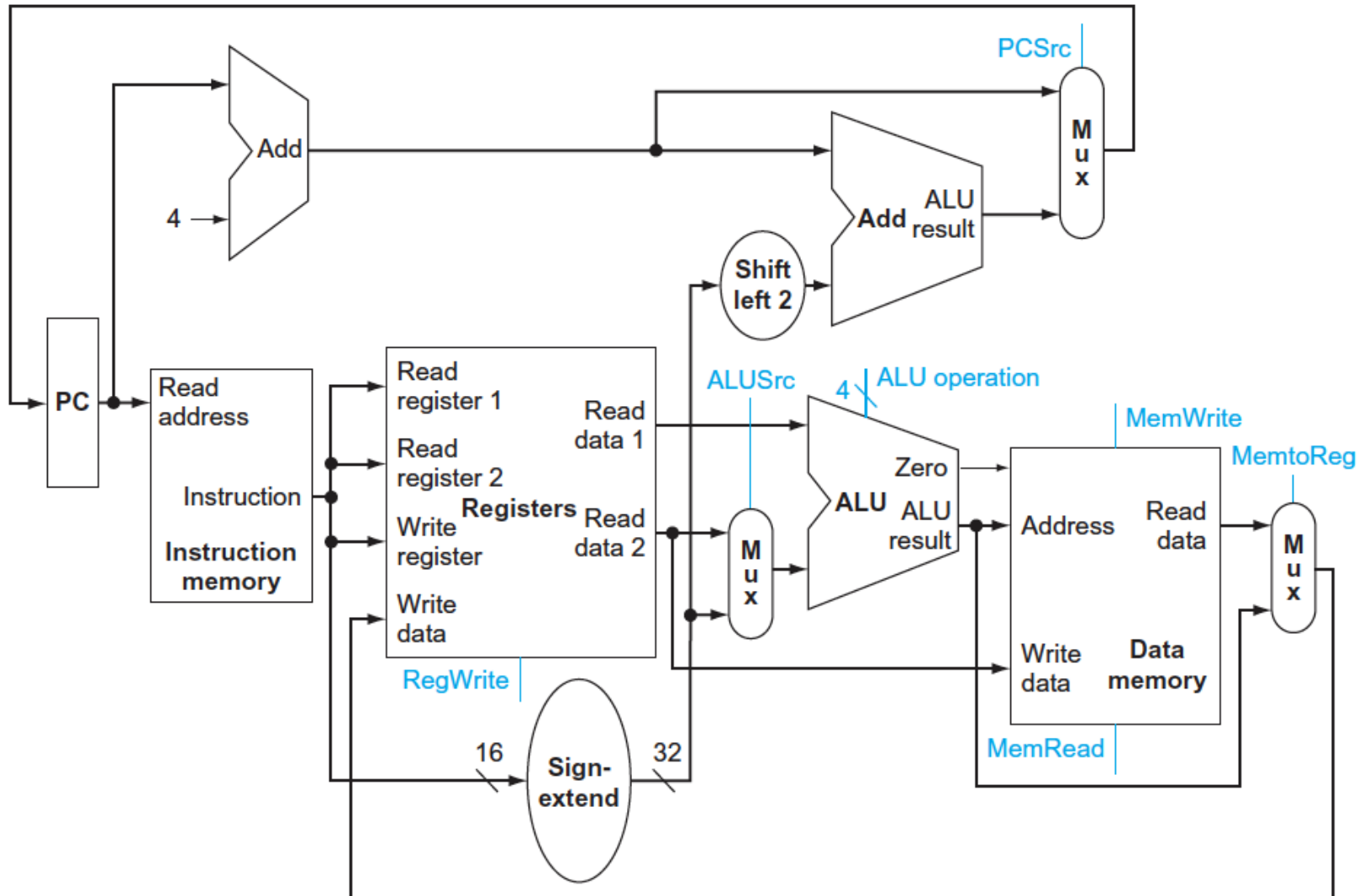
Instrução BEQ

- Instruções Aritméticas

- A instrução contém o **endereço de dois registradores a serem lidos (rs e rt)**
- Estes registradores são lidos e passados para a ALU realizar a operação
- O resultado é **escrito em um terceiro registrador (rd)**

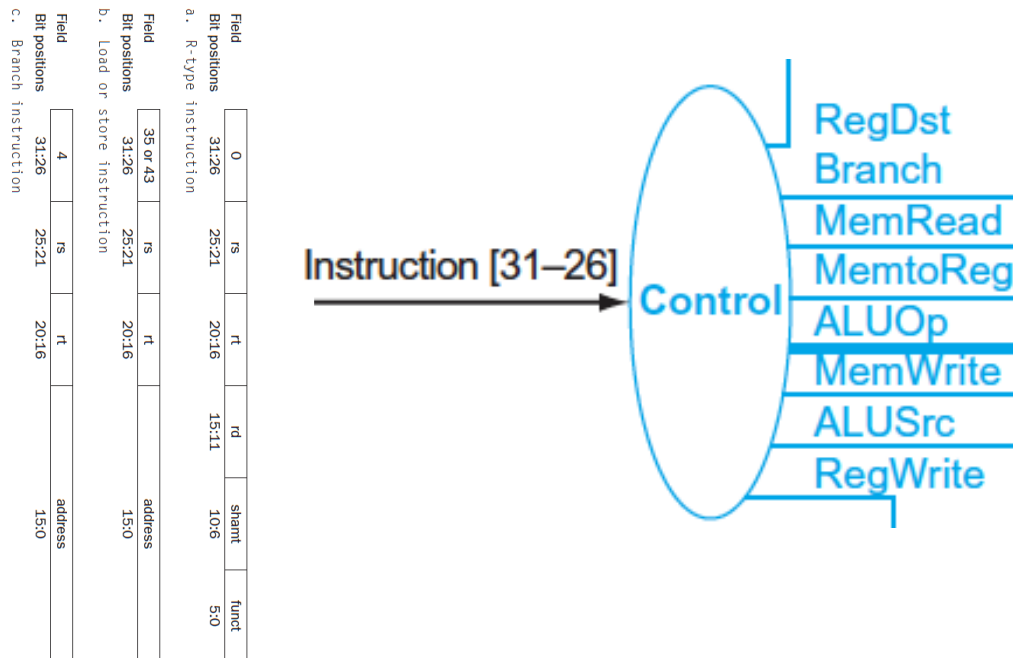


Datapath para ADD,SUB,LW,SW e BEQ



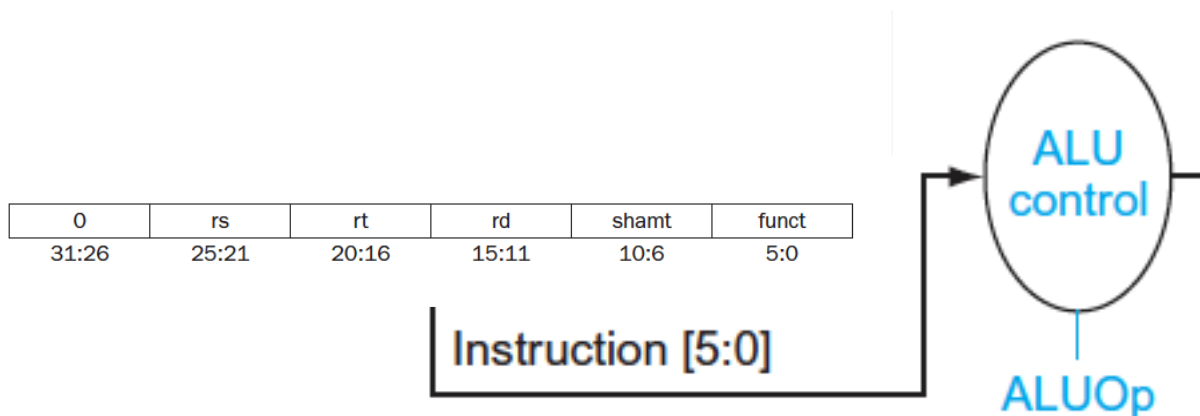
Controle

- Guia a propagação da execução da instrução no datapath
- No MIPS é dividido em 2 blocos: Main e ALU Control
- **Main Control** recebe o **OPCode** e gera todos os sinais para o caminho de dados

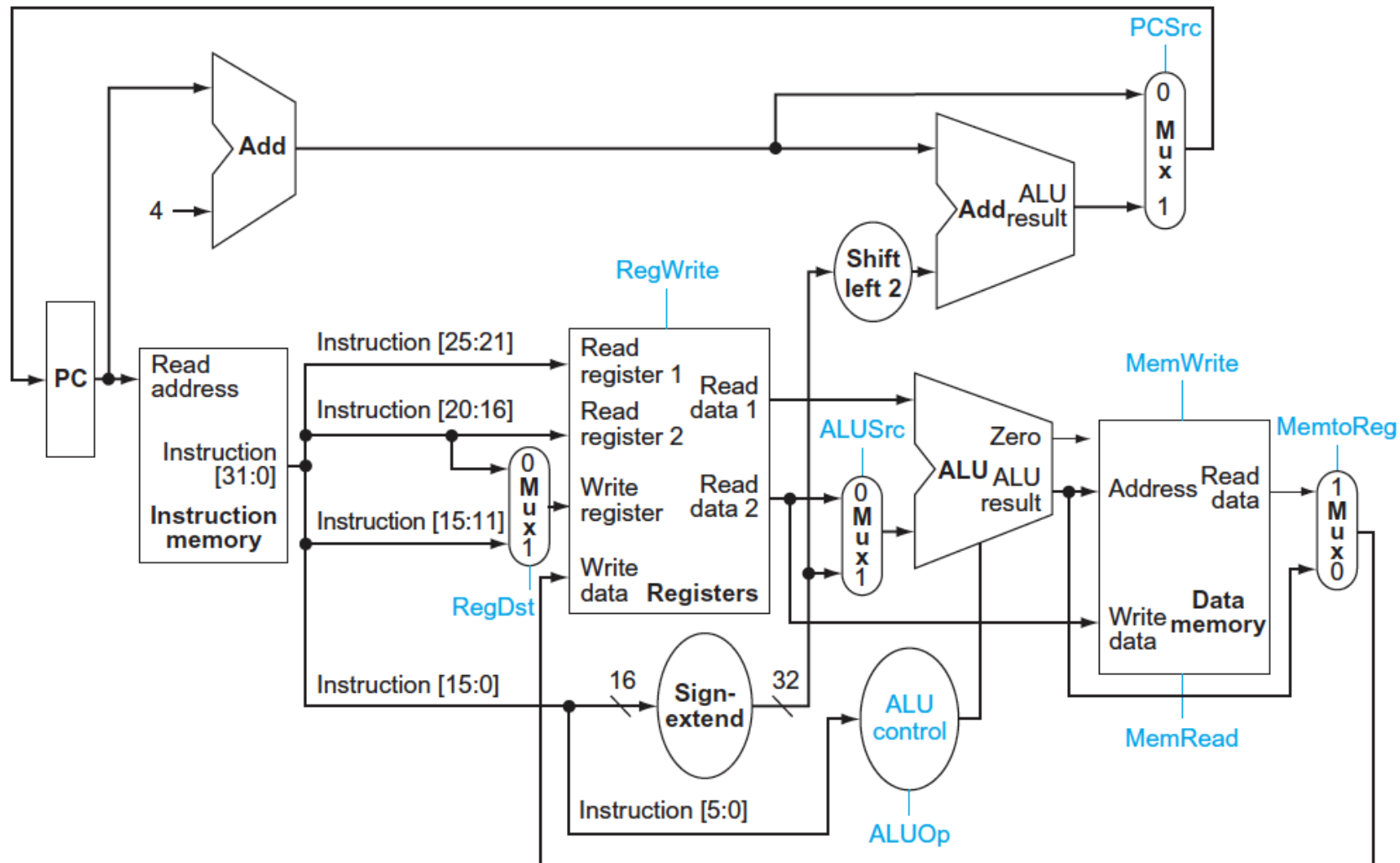


Controle

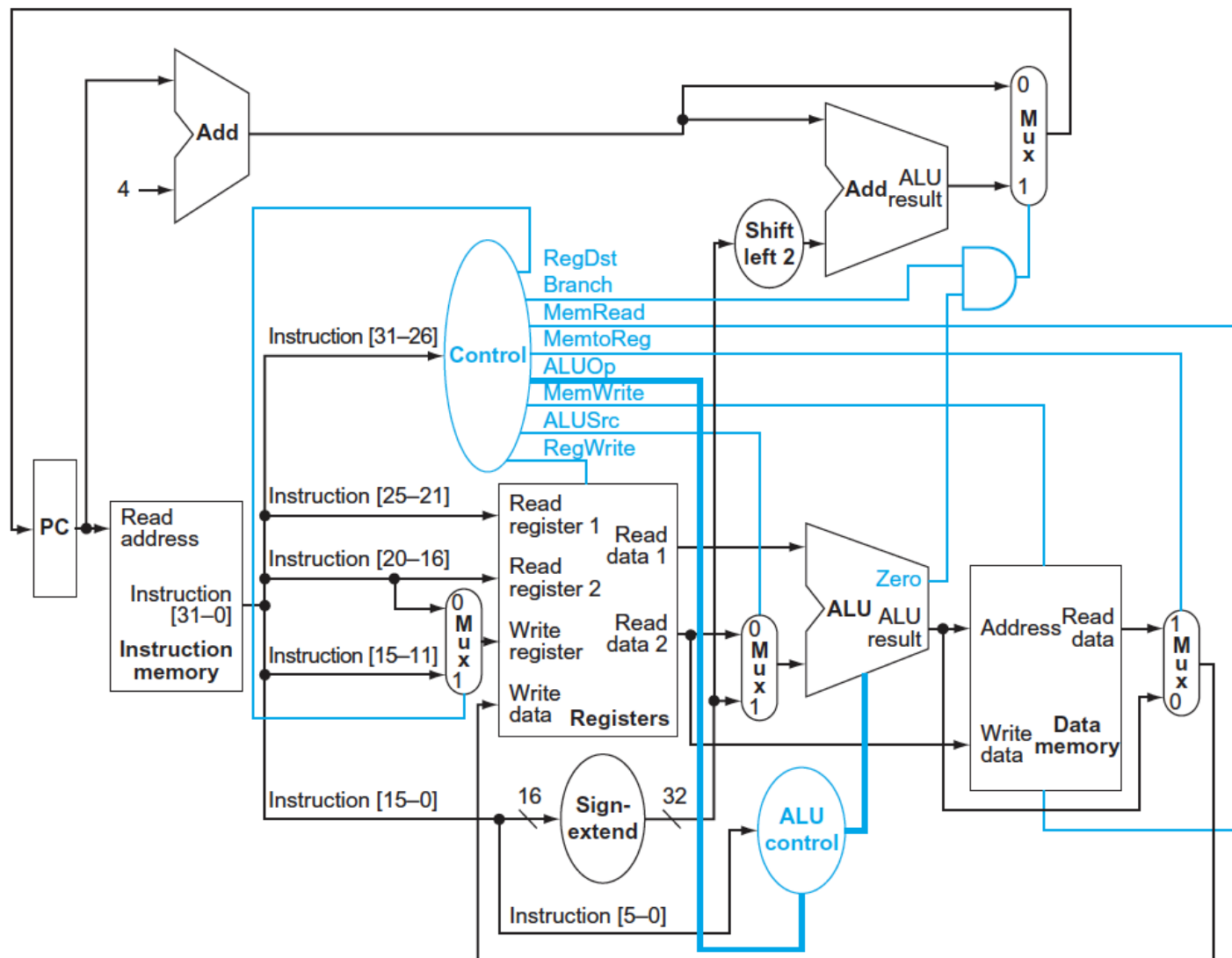
- Guia a propagação da execução da instrução no datapath
- No MIPS é dividido em 2 blocos: Main e ALU Control
- **Para todas as instruções o ALU Control recebe o AluOP gerado pelo Main Control**
- **Para instruções Tipo R o ALU Control recebe o Campo Funct**



Datapath com ALU Control

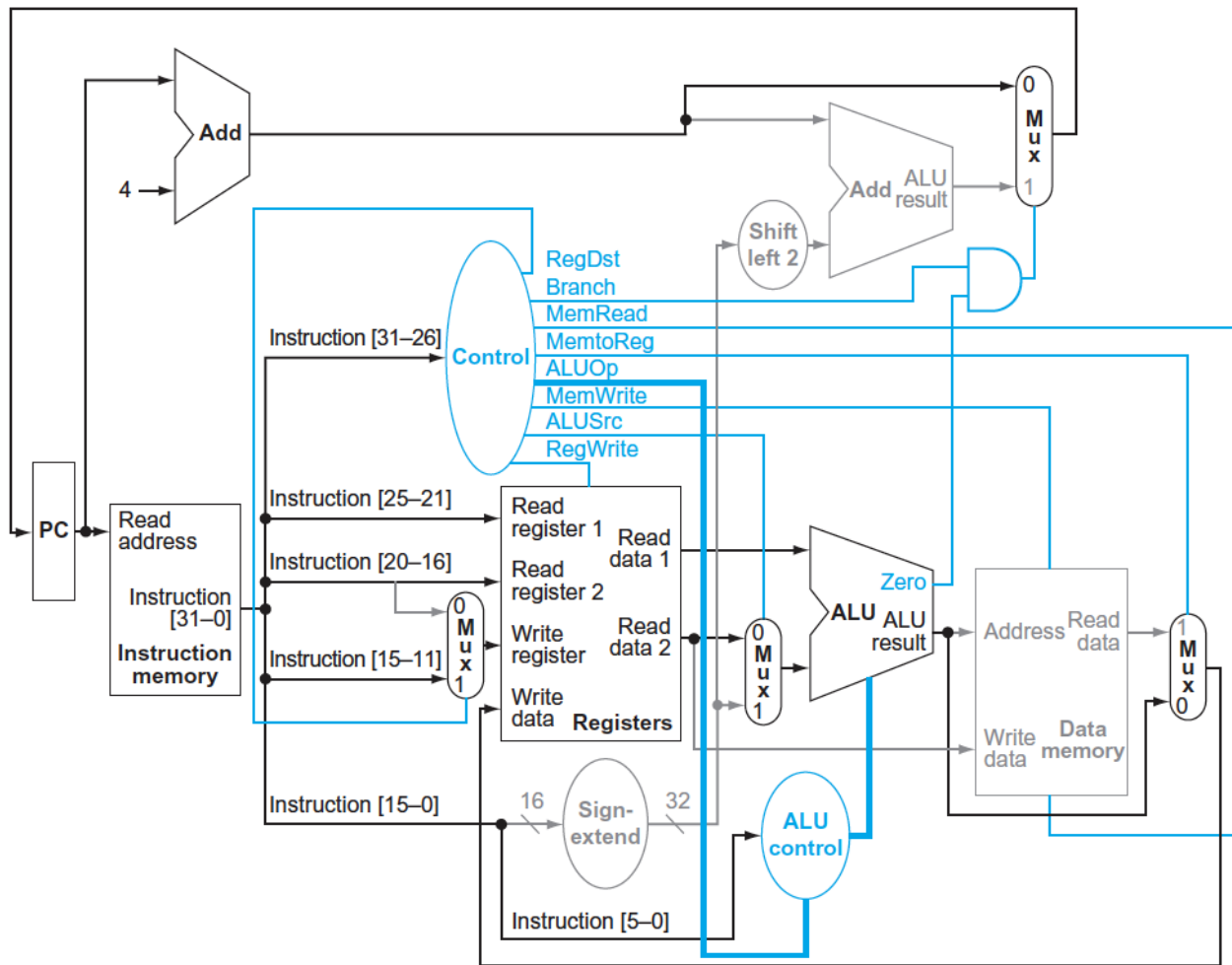


Datapath e Controle



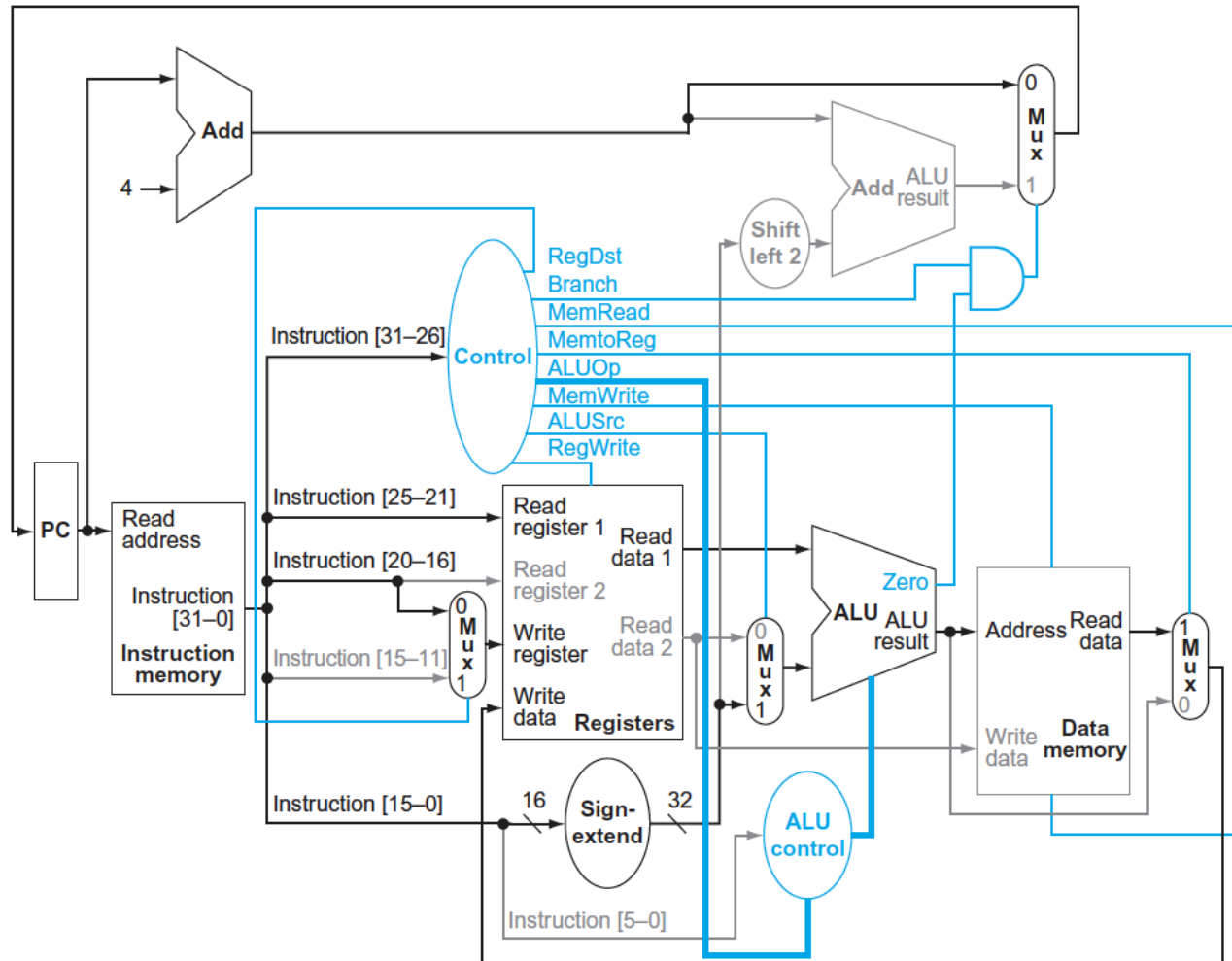
Datapath e Controle

- Execução de ADD ou SUB



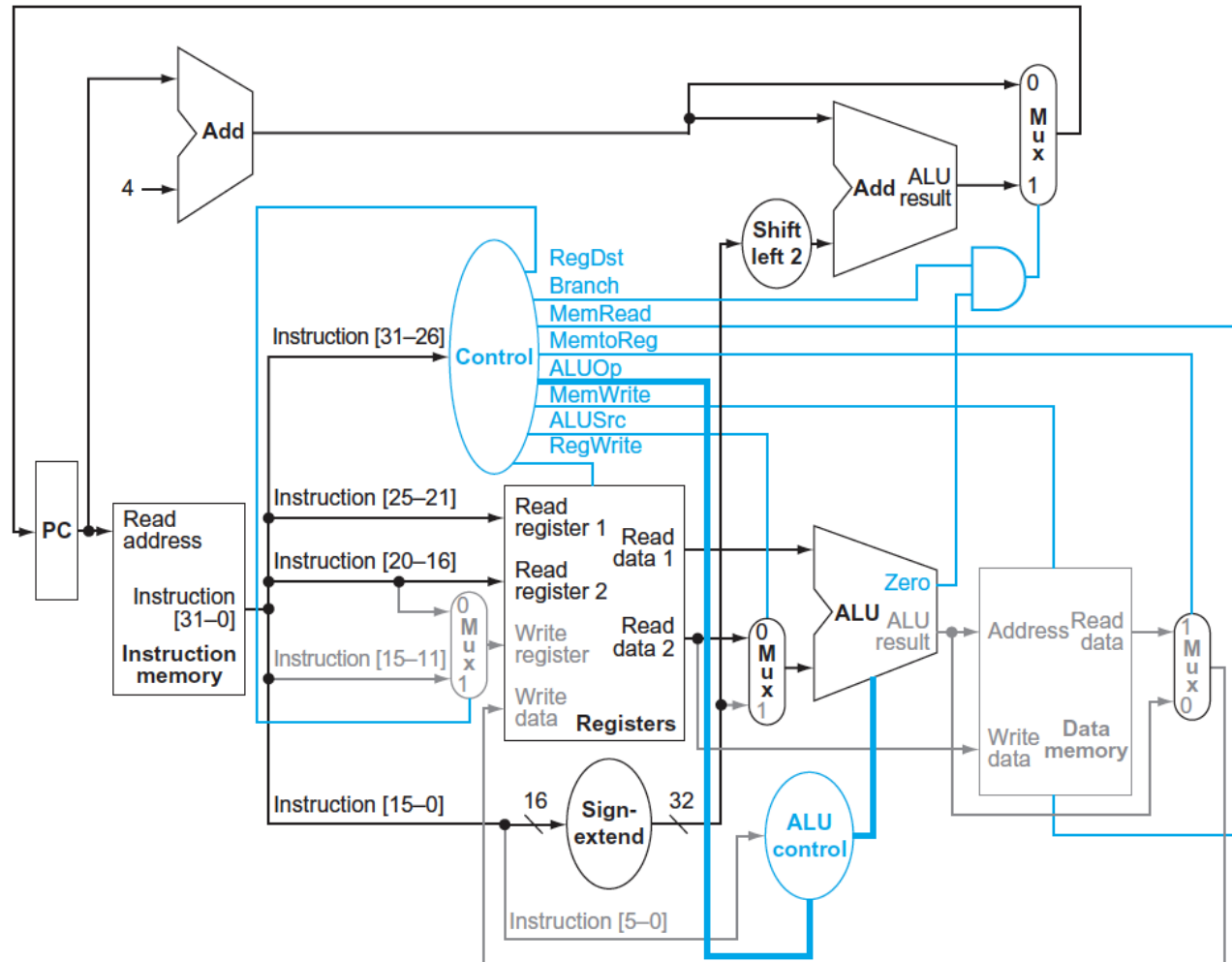
Datapath e Controle

- Execução de LW (Diferença para SW?)



Datapath e Controlo

- Execução de BEQ

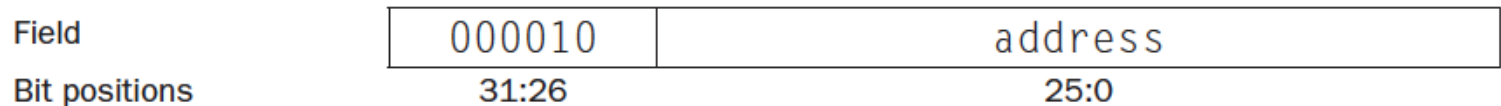


- Tabela Sinais de Entrada e Saída

Input or output	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Implementando Nova Instrução

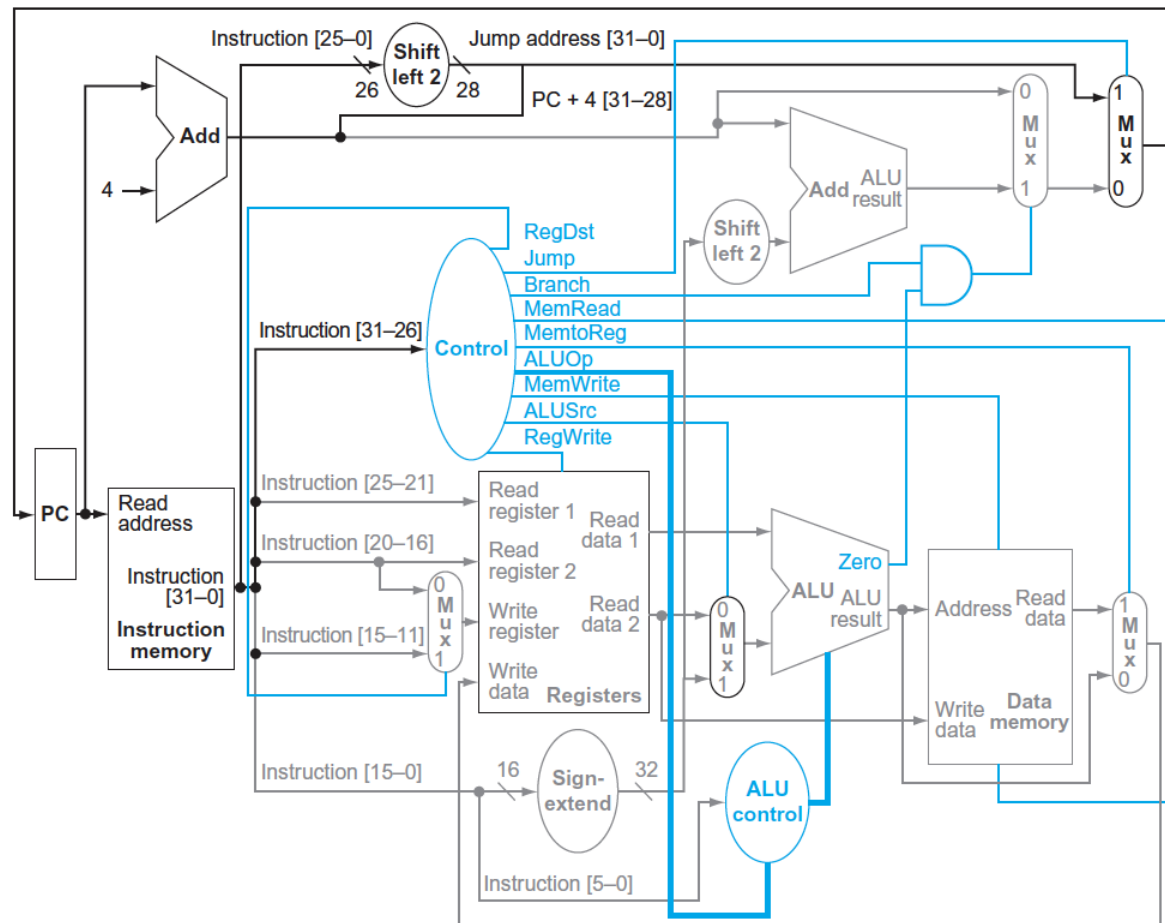
- Desvio Incondicional – Jump
 - Operação no Datapath ($PC \leftarrow PC[31:28] \ \& \ \text{Imed26} \ \& \ "00"$)
 - Formato da Instrução



Implementando Nova Instrução

- Desvio Incondicional – Jump

- Operação no Datapath ($PC \leftarrow PC[31:28] \ \& \ \text{Imed26} \ \& \ "00"$)



- Ao **final de cada ciclo de relógio** o PC é carregado com um novo valor
- Mudança no valor do PC se propaga através de uma **grande lógica combinacional**
- Período do ciclo de relógio deve ser maior do que máximo atraso de propagação através desta lógica combinacional
- Supondo os seguintes atrasos:
 - Memórias: 1 ns
 - ALU: 0.5 ns
 - Banco de registradores: 0.5 ns
 - Somadores: 0.3 ns
 - Demais componentes: atraso desprezível
- Período do ciclo de relógio deve ser maior ou igual a 3.5 ns
 - Frequência: 285 MHz

Exercícios

4.1 Consider the following instruction:

Instruction: `AND Rd, Rs, Rt`

Interpretation: `Reg[Rd] = Reg[Rs] AND Reg[Rt]`

4.1.1 [5] <\$4.1> What are the values of control signals generated by the control in [Figure 4.2](#) for the above instruction?

4.1.2 [5] <\$4.1> Which resources (blocks) perform a useful function for this instruction?

4.1.3 [10] <\$4.1> Which resources (blocks) produce outputs, but their outputs are not used for this instruction? Which resources produce no outputs for this instruction?

4.2 The basic single-cycle MIPS implementation in [Figure 4.2](#) can only implement some instructions. New instructions can be added to an existing Instruction Set Architecture (ISA), but the decision whether or not to do that depends, among other things, on the cost and complexity the proposed addition introduces into the processor datapath and control. The first three problems in this exercise refer to the new instruction:

Instruction: `LWI Rt, Rd(Rs)`

Interpretation: $\text{Reg}[\text{Rt}] = \text{Mem}[\text{Reg}[\text{Rd}] + \text{Reg}[\text{Rs}]]$

4.2.1 [10] <\$4.1> Which existing blocks (if any) can be used for this instruction?

4.2.2 [10] <\$4.1> Which new functional blocks (if any) do we need for this instruction?

4.2.3 [10] <\$4.1> What new signals do we need (if any) from the control unit to support this instruction?

4.3 When processor designers consider a possible improvement to the processor datapath, the decision usually depends on the cost/performance trade-off. In the following three problems, assume that we are starting with a datapath from [Figure 4.2](#), where I-Mem, Add, Mux, ALU, Regs, D-Mem, and Control blocks have latencies of 400 ps, 100 ps, 30 ps, 120 ps, 200 ps, 350 ps, and 100 ps, respectively, and costs of 1000, 30, 10, 100, 200, 2000, and 500, respectively.

Consider the addition of a multiplier to the ALU. This addition will add 300 ps to the latency of the ALU and will add a cost of 600 to the ALU. The result will be 5% fewer instructions executed since we will no longer need to emulate the MUL instruction.

4.3.1 [10] <\$4.1> What is the clock cycle time with and without this improvement?

4.3.2 [10] <\$4.1> What is the speedup achieved by adding this improvement?

4.3.3 [10] <\$4.1> Compare the cost/performance ratio with and without this improvement.

Exercícios

4.4 Problems in this exercise assume that logic blocks needed to implement a processor's datapath have the following latencies:

I-Mem	Add	Mux	ALU	Regs	D-Mem	Sign-Extend	Shift-Left-2
200ps	70ps	20ps	90ps	90ps	250ps	15ps	10ps

4.4.1 [10] <\$4.3> If the only thing we need to do in a processor is fetch consecutive instructions (Figure 4.6), what would the cycle time be?

4.4.2 [10] <\$4.3> Consider a datapath similar to the one in Figure 4.11, but for a processor that only has one type of instruction: unconditional PC-relative branch. What would the cycle time be for this datapath?

4.4.3 [10] <\$4.3> Repeat 4.4.2, but this time we need to support only conditional PC-relative branches.

The remaining three problems in this exercise refer to the datapath element Shift-left-2:

4.4.4 [10] <\$4.3> Which kinds of instructions require this resource?

4.4.5 [20] <\$4.3> For which kinds of instructions (if any) is this resource on the critical path?

4.4.6 [10] <\$4.3> Assuming that we only support beq and add instructions, discuss how changes in the given latency of this resource affect the cycle time of the processor. Assume that the latencies of other resources do not change.